



Lecture 5

Functional Dependencies

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University



[Update Anomalies](#)

[Decomposition](#)

[Functional
Dependencies](#)

[Identifying Functional
Dependency Types](#)

- 1 [Update Anomalies](#)
 - [Decomposition](#)

- 2 [Functional Dependencies](#)

- 3 [Identifying Functional Dependency Types](#)



- What is redundant data and how can it cause problems?
- What is a functional dependency?
- What does *relationships across attributes* mean?
- How can we identify functional dependencies?



- A major aim of relational database design is to group attributes into relations to minimise data redundancy.
- If this aim is achieved, the potential benefits for the implemented database include the following:
 - updates to the data stored in the database are achieved with a minimal number of operations: possibilities for data inconsistencies are reduced;
 - reduction in the file storage space required by base relations: minimising costs.
- Redundancy can be ok: copies of primary keys acting as foreign keys in related relations to enable the modeling of relationships between data.



- We illustrate the problems associated with unwanted data redundancy by comparing the *Staff* and *Branch* relations shown in Figure 1 with the *StaffBranch* relation shown in Figure 2.
- The *StaffBranch* relation is an alternative format of the *Staff* and *Branch* relations.
- The relations have the form:

Staff (*staffNo*, *sName*, *position*, *salary*, *branchNo*)

Branch (*branchNo*, *bAddress*)

StaffBranch (*staffNo*, *sName*, *position*, *salary*, *branchNo*, *bAddress*)

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Figure 1: Staff and Branch Relations

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Figure 2: StaffBranch Relation



- In the *StaffBranch* relation there is redundant data: details of a branch are repeated for every member of staff located at that branch.
- In contrast, branch details appear once for each branch in the *Branch* relation, and only the branch number (*branchNo*) is repeated in the *Staff* relation to represent staff location.
- Relations that have redundant data may have problems called **update anomalies**, which are classified as *insertion*, *deletion*, or *modification* anomalies.

Insertion Anomalies (entering redundant data)



- To insert the details of new members of staff into the *StaffBranch* relation, we must include the details of the branch at which the staff are to be located.
- To insert new staff for branch B007, we must enter the correct details of branch number B007 so that the branch details are consistent with values for branch B007 in other tuples of the *StaffBranch* relation.
- This causes an update anomaly if, for example, branch address is not *exact*.
- Relations in figure 1 do not have this problem: only the branch number for each staff member in the *Staff* relation is entered.
- Details for branch *B007* are recorded as a single tuple in the *Branch* relation.

Insertion Anomalies (entering NULLs)



- To insert a new branch that currently has no members of staff into the *StaffBranch* relation, it is necessary to enter nulls for staff attributes.
- However, as *staffNo* is the primary key for the *StaffBranch* relation, attempting to enter nulls for *staffNo* violates entity.
- We cannot enter a tuple for a new branch with no staff members!
- The relations in Figure 1 avoid this problem because branch details are entered in the *Branch* relation separately from the staff details.
- Details of staff ultimately located at that branch are entered at a later date into the *Staff* relation.



- If we delete a tuple from the *StaffBranch* relation that represents the last member of staff located at a branch, the details about that branch are also lost from the database.
- Delete the tuple for staff number SA9 (Mary Howe) from the *StaffBranch* relation and details relating to branch number B007 are lost.
- The design of the relations in Figure 1 avoids this problem, because branch tuples are stored separately from staff tuples, with *branchNo* joining the two relations.
- If we delete the tuple for staff number SA9 from the *Staff* relation, details for branch B007 remain unaffected in the *Branch* relation.



- If we must change the value of one attribute of a particular branch in the *StaffBranch* relation (address for branch number B003), we must update the tuples of all staff located at that branch.
- If this modification is not carried out on all appropriate tuples in *StaffBranch*, the database will become **inconsistent**.
- In this example, branch number B003 may appear to have different addresses in different staff tuples.



- These examples illustrate that the *Staff* and *Branch* relations in Figure 1 have more desirable properties than the *StaffBranch* relation in Figure 2.
- This demonstrates that while the *StaffBranch* relation is subject to update anomalies, we can avoid these anomalies by **decomposing** the original relation into the *Staff* and *Branch* relations.



There are two important properties associated with decomposition of a larger relation into smaller relations:

- The **lossless-join property** ensures that any instance of the original relation can be identified from corresponding instances in the smaller relations.
- The **dependency preservation property** ensures that a constraint on the original relation can be maintained by enforcing constraints on *each* of the smaller relations. Thus, it is not necessary to perform joins on the smaller relations to check whether a constraint on the original relation is violated.

Functional Dependency

If A and B are attributes of relation R , B is functionally dependent on A (denoted $A \rightarrow B$), if each value of A is associated with exactly one value of B .

- An FD describes the relationship between attributes in a relation and A and B may each consist of one or more attributes.
- **Functional dependency** is a property of the meaning or semantics of the attributes in a relation.
- The semantics indicate how attributes relate to one another, and specify the functional dependencies between attributes.
- When a functional dependency is present, the dependency is specified as a *constraint* between the attributes.





- Consider a relation with attributes A and B , where attribute B is functionally dependent on attribute A .
- If we know the value of A and examine the relation with this dependency: we find only one value of B in all tuples that have a given value of A , at any moment in time.
- Thus, when two tuples have the same value of A , they also have the same value of B : but for any B , there may be different values of A !
- An alternative way to describe the relationship between attributes A and B is to say:
 A functionally determines B .
- The dependency between attributes A and B can be represented diagrammatically, as shown in Figure 3.

Determinant

Refers to the attribute, or group of attributes, on the left-hand side of a functional dependency.

- When a functional dependency exists, the attribute or group of attributes on the left-hand side of the arrow is called the **determinant**.
- For example, in Figure 3, *A* is the determinant of *B*.

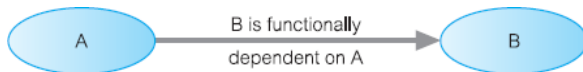


Figure 3: Functional dependency diagram.



- Consider the attributes *staffNo* and *position* in the *Staff* relation in Figure 1.
- For a specific *staffNo* (e.g. SL21), one can determine the position of that staff member as *Manager*.
- Here, *staffNo* functionally determines *position*, as shown in Figure 4(a).
- However, Figure 4(b) illustrates that the opposite is not true, as *position* does not functionally determine *staffNo*.
- A member of staff holds *one* position: but several members of staff may have the same position.
- The relationship between *staffNo* and *position* is one-to-one (1:1): for each staff number, there is only one position.

- On the other hand, the relationship between *position* and *staffNo* is one-to-many (1:*): there are several staff numbers associated with a given position.
- Here, *staffNo* is the determinant of this FD.

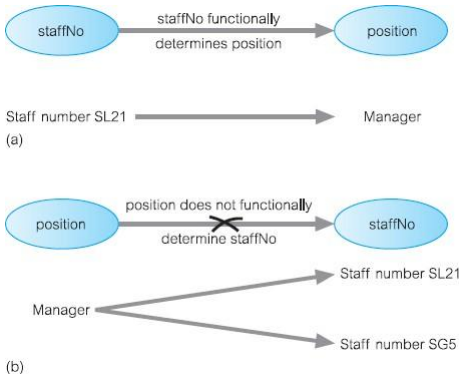


Figure 4: (a) $\text{staffNo} \rightarrow \text{position}$... (b) $\text{position} \not\rightarrow \text{staffNo}$



- **Aim:** identify functional dependencies between attributes that have a *one-to-one* relationship between attribute(s) on the left-hand side (determinant), and attribute(s) on the right-hand side of a dependency.
- When identifying FDs between attributes, it is important to distinguish between the values held by an attribute at a *given point in time* and the set of all possible values that an attribute may hold at *different times*.
- In other words, a functional dependency is a property of a relational *schema* (intension) and not a property of a particular *instance* of the schema (extension).



- Consider the values shown in *staffNo* and *sName* attributes of the *Staff* relation in Figure 1.
- For a specific *staffNo* (SL21), we can determine the name of that member of staff as 'John White'.
- Furthermore, it appears that for a specific *sName* (eg. John White), we can determine the staff number for that member as 'SL21'.
- Can we conclude that *staffNo* determines *sName* and/or that *sName* determines *staffNo*?
- If the values of the *Staff* relation in Figure 1 represent the set of *all* possible values for *staffNo* and *sName* attributes, then these FDs hold:

$\text{staffNo} \rightarrow \text{sName}$

$\text{sName} \rightarrow \text{staffNo}$



- However, if the values shown in the *Staff* relation simply represent a set of values for *staffNo* and *sName* attributes at a *given moment in time*, then we are not interested in these relationships.
- Why? We want to identify functional dependencies that hold for *all* possible values for attributes as these represent the types of **integrity constraints** that must be identified.
- Such constraints indicate the *limitations* on the *values* that a relation can legitimately assume.



- One approach to identifying the set of all possible values for attributes in a relation, is to more clearly understand the purpose of each attribute.
- The *staffNo* attribute is to uniquely identify each member of staff, whereas the purpose of the values held in the *sName* attribute is to hold the names of members of staff.
- If we know the staff number (*staffNo*) of a member of staff, we can determine the name of the member of staff (*sName*) remains true.
- However, as *sName* may hold duplicate values for members of staff, then for some members of staff we would be unable to determine their staff number (*staffNo*).

FDs represent 1-1 relationships



- The relationship between *staffNo* and *sName* is 1 – to – 1: for each staff number there is only one name.
- The relationship between *sName* and *staffNo* is 1 – to – N: there may be several staff numbers associated with a given name.
- The functional dependency that remains true after consideration of all possible values for the *staffNo* and *sName* attributes of the *Staff* relation is:

$\text{staffNo} \rightarrow \text{sName}$

Full Functional Dependency (Example)



- Consider the following functional dependency that exists in the *Staff* relation of Figure 1:
 $\text{staffNo}, \text{sName} \rightarrow \text{branchNo}$
- It is correct to say that each value of $(\text{staffNo}, \text{sName})$ is associated with a single value of *branchNo*.
- However, it is not a **full functional dependency** because *branchNo* is also functionally dependent on a subset of $(\text{staffNo}, \text{sName})$: *staffNo*.
- In other words, the functional dependency shown above is an example of a **partial dependency**.
- The type of functional dependency that we are interested in identifying is the **full functional dependency**:
 $\text{staffNo} \rightarrow \text{branchNo}$



In summary, the functional dependencies that we use in **normalisation** have the following characteristics:

- 1 There is a *one-to-one* relationship between the attribute(s) on the left-hand side (determinant) and those on the right-hand side of a functional dependency.
- 2 They hold for all time.
- 3 The **determinant** has the *minimal* number of attributes necessary to maintain the dependency with the attribute(s) on the right-hand side.

In other words, there must be a **full functional dependency** between the attribute(s) on the left- and right-hand sides of the dependency.



Transitive Dependency

A condition where A , B , and C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).

- Consider the following functional dependencies within the *StaffBranch* relation from Figure 2:
 $staffNo \rightarrow sName, position, salary, branchNo, bAddress$
 $branchNo \rightarrow bAddress$
- The transitive dependency $branchNo \rightarrow bAddress$ exists on $staffNo$ via $branchNo$.
- In other words, $staffNo$ functionally determines $bAddress$ via the $branchNo$ attribute and neither $branchNo$ nor $bAddress$ functionally determines $staffNo$.



Identifying a set of FDs for *StaffBranch*

- Begin by examining the semantics of attributes in the *StaffBranch* relation from Figure 2.
- Assume that the position held and the branch determine a member of staff's salary.
- Identify FDs based on an understanding of the attributes and for each FD, ensure that all attributes on the right-hand side are functionally dependent on the determinant on the left-hand side.

`staffNo` $\rightarrow$ `sName, position, salary, branchNo, bAddress`

`branchNo` $\rightarrow$ `bAddress`

`bAddress` $\rightarrow$ `branchNo`

`branchNo, position` $\rightarrow$ `salary`

`bAddress, position` $\rightarrow$ `salary`



- Consider identifying FDs with no information about the meaning of attributes and their relationships.
- One can identify FDs using sample data if it is a **true representation** of all possible data values that the database may hold.
- Consider the data for attributes *A*, *B*, *C*, *D*, and *E* in the *Sample* relation in Figure 5.
- Assume that the values shown in this relation are representative of all possible values that can be held by attributes *A*, *B*, *C*, *D*, and *E*.
- The process of identifying FDs (denoted fd1 to fd4) that exist between the attributes of the *Sample* relation shown in Figure 5 is now described.

Using Sample Data to Identify FDs

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t

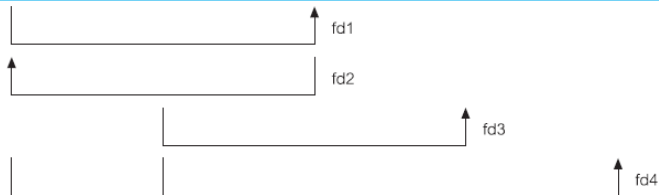


Figure 5: The Sample relation displaying data for attributes A, B, C, D, and E and the functional dependencies (fd1 to fd4) that exist between these attributes.



- **Approach.** To identify the functional dependencies that exist between attributes A, B, C, D, and E, we examine the Sample relation shown in Figure 5 and identify when values in one column are consistent with the presence of particular values in other columns.

- **Algorithm.**

- Begin with the first column
- Work our way over to the right-hand side of the relation (taking each attribute in turn)
- Take combinations of columns (where values in two or more columns are consistent with the appearance of values in other columns).



- Here, when the value 'a' appears in column A the value 'z' appears in column C , and when 'e' appears in column A the value 'r' appears in column C .
- We conclude that there is a 1 – to – 1 relationship between attributes A and C .
- In other words, $A \rightarrow C$ and this is shown as functional dependency 1 (fd1) in Figure 5.
- Also, the values in column C are consistent with specific values in column A , so we conclude that there is a 1 – to – 1 relationship between attributes C and A .
- Thus, $C \rightarrow A$ and is shown as fd2 in Figure 5.



- Now consider attribute B , we can see that when 'b' or 'd' appears in column B then 'w' appears in column D and when 'f' appears in column B then 's' appears in column D .
- We conclude that there is a 1 – to – 1 relationship between B and D .
- In other words, $B \rightarrow D$ and this is shown as fd3 in Figure 5.
- However, attribute D does not functionally determine attribute B as a single unique value in column D is not associated with a single value in column B .
- Specifically, when 'w' appears in column D , either 'b' or 'd' could appear in B .
- Thus, there is a one-to-many relationship between attributes D and B .



- The final single attribute to consider is E but the values are not associated with the consistent appearance of particular values in the other columns.
- In other words, attribute E does not functionally determine attributes A , B , C , or D .



- Now consider *combinations of attributes* and the appearance of consistent values in other columns.
- Unique combination of values in columns A and B such as (a,b) is associated with a single value in column E , which in this example is 'q'.
- In other words, $(A, B) \rightarrow E$ and this is shown as fd4 in Figure 5.
- However, the reverse is not true: we previously stated that attribute E does not functionally determine any other attribute in the relation.



- We complete the examination of the relation in Figure 5 by considering *all* remaining combinations of columns.
- In summary, we describe the FDs between attributes *A* to *E* in the *Sample* relation as follows:

$A \rightarrow C$ (fd1)

$C \rightarrow A$ (fd2)

$B \rightarrow D$ (fd3)

$A, B \rightarrow E$ (fd4)



- Every rule $A \rightarrow B$ must have $|A| \geq 1$ and $|B| \geq 1$ and A and B are disjoint.
- Start simple with rules where: $|A| = 1$ and $|B| = 1$.
- Start simple with rules where: $|A| = 1$ and $|B| > 1$.
- Move to rules where: $|A| = 2$ and $|B| = 1$.
- Move to rules where: $|A| = 2$ and $|B| > 1$.
- If no. of columns = n , eventually arrive at: $|A| = n - 1$ and $|B| = 1$.

Identify the Primary Key using FDs



- The main purpose of identifying a set of functional dependencies for a relation is to specify the set of **integrity constraints** that *must* hold on a relation.
- Consider first the identification of candidate keys, one of which is selected to be the primary key for the relation.
- Focus on the identification of five FDs for the *StaffBranch* relation shown in Figure 2.
- The determinants for these FDs are *staffNo*, *branchNo*, *bAddress*, *(branchNo, position)*, and *(bAddress, position)*.



- To identify the candidate key(s) for the *StaffBranch* relation, find the attribute (or group of attributes) that uniquely identifies each tuple in the relation.
- If a relation has more than one candidate key, identify the primary key.
- Attributes that are not part of the primary key (non-primary-key attributes) **should be functionally dependent** on the key.
- The sole candidate key for *StaffBranch* and therefore the primary key, is *staffNo*, as *all other attributes* are functionally dependent on *staffNo*.
- Although *branchNo*, *bAddress*, (*branchNo*, *position*), and (*bAddress*, *position*) are determinants in this relation, they are *not* candidate keys for the relation.

Identify the primary key for Sample



- From the four FDs for *Sample*, we examine the *determinant* for each FD to identify the candidate key(s) for the relation.
- A suitable determinant must functionally determine the other attributes in the relation.
- The determinants in the *Sample* relation are A , B , C , and (A, B) .
- However, the only determinant that functionally determines *all other attributes* of the relation is (A, B) .
- Specifically, $A \rightarrow C$, $B \rightarrow D$, and $(A, B) \rightarrow E$.
- In other words, the attributes that make up the determinant (A, B) can determine all the other attributes in the relation either separately as A or B or together as (A, B) .



- Thus, an essential characteristic for a candidate key is that the attributes of a determinant either *individually* or *working together* must functionally determine **all other attributes** in the relation.
- This is not a characteristic of the other determinants in the *Sample* relation (*A*, *B*, or *C*) as in each case they determine only one other attribute in the relation.
- As there are no other candidate keys for the *Sample* relation, (*A*, *B*) is identified as the primary key for this relation.



[Update Anomalies](#)

[Decomposition](#)

[Functional
Dependencies](#)

[Identifying Functional
Dependency Types](#)

